

LAB ACTIVITY (Lab-08)

1. Banking System with Transactions

Features & Concepts Covered

1. **Classes**
 - **Account** (base class), **SavingsAccount**, **CurrentAccount** (subclasses).
2. **Constructors**
 - **Default + Parameterized constructors.**
3. **Inheritance & Overriding**
 - **SavingsAccount** and **CurrentAccount** override **withdraw()** with different rules.
4. **Overloading Methods**
 - **deposit(int amount)** vs **deposit(double amount)**.
5. **Interfaces**
 - **Transaction** with **deposit()** and **withdraw()**.
 - **Auditable** (marker interface).
6. **Abstract Class**
 - **BankUser** with abstract **showDetails()**.
7. **Use of this keyword**
 - **Inside constructors to refer to current object.**
8. **Access Modifiers**
 - **Private balance, protected methods, public service methods.**
9. **Control Statements**
 - **if-else** for transaction logic.
 - **switch** for menu-driven program.
 - **while/do-while** for continuous banking session.
 - **break and continue** for exiting operations.
10. **Packages**
 - **bank.accounts, bank.transactions, bank.users.**
11. **Arrays**
 - **Store multiple accounts in an array or ArrayList.**
12. **Final Keyword**
 - **A final constant like MIN_BALANCE = 500.**

```
import java.util.*;

// Abstract class
abstract class BankUser {
    protected String name;
    protected int accountNumber;

    public BankUser(String name, int accountNumber) {
        this.name = name;
        this.accountNumber = accountNumber;
    }

    public abstract void showDetails();
}

// Interface
interface Transaction {
    void deposit(double amount);
    void withdraw(double amount);
}
```

```

}

// Marker interface
interface Auditable {}

// Base Account class
class Account extends BankUser implements Transaction, Auditable {
    protected double balance;
    protected static final double MIN_BALANCE = 500.0;

    public Account(String name, int accountNumber, double balance) {
        super(name, accountNumber);
        this.balance = balance;
    }

    // Overloading deposit()
    public void deposit(int amount) {
        deposit((double) amount);
    }

    @Override
    public void deposit(double amount) {
        this.balance += amount;
        System.out.println("Deposited: " + amount);
    }

    @Override
    public void withdraw(double amount) {
        if (balance - amount >= MIN_BALANCE) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient balance! Minimum balance must be maintained.");
        }
    }

    @Override
    public void showDetails() {
        System.out.println("Account Holder: " + name);
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Balance: " + balance);
    }
}

// SavingsAccount inherits from Account
class SavingsAccount extends Account {
    public SavingsAccount(String name, int accountNumber, double balance) {
        super(name, accountNumber, balance);
    }

    @Override
    public void withdraw(double amount) {
        System.out.println("Savings Account Withdrawal:");
        super.withdraw(amount);
    }
}

// CurrentAccount inherits from Account
class CurrentAccount extends Account {

```

```

private static final double OVERDRAFT_LIMIT = -1000.0;

public CurrentAccount(String name, int accountNumber, double balance) {
    super(name, accountNumber, balance);
}

@Override
public void withdraw(double amount) {
    if (balance - amount >= OVERDRAFT_LIMIT) {
        balance -= amount;
        System.out.println("Withdrawn: " + amount);
    } else {
        System.out.println("Overdraft limit exceeded!");
    }
}
}

// Main menu-driven program
public class BankingSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Account> accounts = new ArrayList<>();
        int choice;

        do {
            System.out.println("\n=== Banking System Menu ===");
            System.out.println("1. Create Savings Account");
            System.out.println("2. Create Current Account");
            System.out.println("3. Deposit");
            System.out.println("4. Withdraw");
            System.out.println("5. Show Account Details");
            System.out.println("6. Search Account by Name");
            System.out.println("7. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    System.out.print("Enter Name: ");
                    sc.nextLine();
                    String sName = sc.nextLine();
                    System.out.print("Enter Account Number: ");
                    int sAccNo = sc.nextInt();
                    System.out.print("Enter Initial Balance: ");
                    double sBal = sc.nextDouble();
                    accounts.add(new SavingsAccount(sName, sAccNo, sBal));
                    System.out.println("Savings Account Created!");
                    break;

                case 2:
                    System.out.print("Enter Name: ");
                    sc.nextLine();
                    String cName = sc.nextLine();
                    System.out.print("Enter Account Number: ");
                    int cAccNo = sc.nextInt();
                    System.out.print("Enter Initial Balance: ");
                    double cBal = sc.nextDouble();
                    accounts.add(new CurrentAccount(cName, cAccNo, cBal));
                    System.out.println("Current Account Created!");

```

```

        break;

    case 3:
        System.out.print("Enter Account Number: ");
        int dAccNo = sc.nextInt();
        for (Account acc : accounts) {
            if (acc.accountNumber == dAccNo) {
                System.out.print("Enter Deposit Amount: ");
                double damt = sc.nextDouble();
                acc.deposit(damt);
            }
        }
        break;

    case 4:
        System.out.print("Enter Account Number: ");
        int wAccNo = sc.nextInt();
        for (Account acc : accounts) {
            if (acc.accountNumber == wAccNo) {
                System.out.print("Enter Withdraw Amount: ");
                double wamt = sc.nextDouble();
                acc.withdraw(wamt);
            }
        }
        break;

    case 5:
        System.out.print("Enter Account Number: ");
        int sAccDetails = sc.nextInt();
        for (Account acc : accounts) {
            if (acc.accountNumber == sAccDetails) {
                acc.showDetails();
            }
        }
        break;

    case 6:
        System.out.print("Enter Name to Search: ");
        sc.nextLine();
        String searchName = sc.nextLine();
        boolean found = false;
        for (Account acc : accounts) {
            if (acc.name.equalsIgnoreCase(searchName)) {
                acc.showDetails();
                found = true;
            }
        }
        if (!found) {
            System.out.println("No account found with the name: " + searchName);
        }
        break;

    case 7:
        System.out.println("Exiting... Thank you!");
        break;

    default:
        System.out.println("Invalid choice!");
}

```

```

    } while (choice != 7);

    sc.close();
}
}

```

```

PS D:\SEMESTER I\LAB WORK\Java Programming\lab8(2709)> cd "d:\SEMESTER I\LAB WORK\Java Programming\lab8(2709)" & java BankingSystem

```

```

=== Banking System Menu ===

```

1. Create Savings Account
2. Create Current Account
3. Deposit
4. Withdraw
5. Show Account Details
6. Search Account by Name
7. Exit

```
Enter choice: 1
```

```
Enter Name: Aakash
```

```
Enter Account Number: 100
```

```
Enter Initial Balance: 6000
```

```
Savings Account Created!
```

```

=== Banking System Menu ===

```

1. Create Savings Account
2. Create Current Account
3. Deposit
4. Withdraw
5. Show Account Details
6. Search Account by Name
7. Exit

```
Enter choice: 2
```

```
Enter Name: Bhawani
```

```
Enter Account Number: 101
```

```
Enter Initial Balance: 5000
```

```
Current Account Created!
```

```

=== Banking System Menu ===

```

1. Create Savings Account
2. Create Current Account
3. Deposit
4. Withdraw
5. Show Account Details
6. Search Account by Name
7. Exit

```
Enter choice: 3
```

```
Enter Account Number: 100
```

```
Enter Deposit Amount: 4000
```

```
Deposited: 4000.0
```

```
=== Banking System Menu ===
```

1. Create Savings Account
2. Create Current Account
3. Deposit
4. Withdraw
5. Show Account Details
6. Search Account by Name
7. Exit

```
Enter choice: 4
```

```
Enter Account Number: 101
```

```
Enter Withdraw Amount: 1000
```

```
Withdrawn: 1000.0
```

```
=== Banking System Menu ===
```

1. Create Savings Account
2. Create Current Account
3. Deposit
4. Withdraw
5. Show Account Details
6. Search Account by Name
7. Exit

```
Enter choice: 5
```

```
Enter Account Number: 100
```

```
Account Holder: Aakash
```

```
Account Number: 100
```

```
Balance: 10000.0
```

```
=== Banking System Menu ===
```

1. Create Savings Account
2. Create Current Account
3. Deposit
4. Withdraw
5. Show Account Details
6. Search Account by Name
7. Exit

```
Enter choice: 5
```

```
Enter Account Number: 101
```

```
Account Holder: Bhawani
```

```
Account Number: 101
```

```
Balance: 4000.0
```

```
=== Banking System Menu ===
```

1. Create Savings Account
2. Create Current Account
3. Deposit
4. Withdraw
5. Show Account Details
6. Search Account by Name
7. Exit

```
Enter choice: 6
```

```
Enter Name to Search: Aakash
```

```
Account Holder: Aakash
```

```
Account Number: 100
```

```
Balance: 10000.0
```

```
=== Banking System Menu ===
```

1. Create Savings Account
2. Create Current Account
3. Deposit
4. Withdraw
5. Show Account Details
6. Search Account by Name
7. Exit

```
Enter choice: 7
```

```
Exiting... Thank you!
```

```
PS D:\SEMESTER I\LAB WORK\Java Programming\lab8(2709)> 
```

2. University Management System

- Abstract class → **Person**
- Inheritance & Overriding → **Student, Professor**
- Interfaces → **Enrollable**
- Marker interface (in Banking System project)
- Constructors (default + parameterized)
- **this** keyword (inside constructors)
- Access Modifiers (private, protected, public)
- Final class with constants → **UniversityConstants**
- Control statements (switch, if-else, loops, break, continue)
- Arrays/ArrayLists for object storage
- Menu-driven program

```
import java.util.*;

// Abstract class
abstract class Person {
    protected String name;
    protected int id;

    public Person() {
        this("Unknown", 0);
    }

    public Person(String name, int id) {
        this.name = name;
        this.id = id;
    }

    public abstract void showDetails();
}

// Interface
interface Enrollable {
    void enroll(String course);
}

// Marker interface for University Management System
interface UniversityMember {}

// Final class with constants
final class UniversityConstants {
    public static final String UNIVERSITY_NAME = "ABC University";
    public static final int MAX_COURSES = 5;
    private UniversityConstants() {}
}

// Student class
class Student extends Person implements Enrollable, UniversityMember {
    private ArrayList<String> courses;

    public Student() {
        this("Unknown Student", 0);
    }
}
```

```

public Student(String name, int id) {
    super(name, id);
    this.courses = new ArrayList<>();
}

@Override
public void enroll(String course) {
    if (courses.size() < UniversityConstants.MAX_COURSES) {
        courses.add(course);
        System.out.println(name + " enrolled in " + course);
    } else {
        System.out.println("Cannot enroll in more than " + UniversityConstants.MAX_COURSES + "
courses.");
    }
}

@Override
public void showDetails() {
    System.out.println("Student Name: " + name);
    System.out.println("Student ID: " + id);
    System.out.println("Courses: " + courses);
}
}

// Professor class
class Professor extends Person implements Enrollable, UniversityMember {
    private String department;
    private ArrayList<String> courses;

    public Professor() {
        this("Unknown Professor", 0, "Unknown");
    }

    public Professor(String name, int id, String department) {
        super(name, id);
        this.department = department;
        this.courses = new ArrayList<>();
    }

    @Override
    public void enroll(String course) {
        courses.add(course);
        System.out.println(name + " assigned to teach " + course);
    }

    @Override
    public void showDetails() {
        System.out.println("Professor Name: " + name);
        System.out.println("Professor ID: " + id);
        System.out.println("Department: " + department);
        System.out.println("Courses Teaching: " + courses);
    }
}

public class UniversityManagement {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Person> people = new ArrayList<>();
    }
}

```

```

int choice;
do {
    System.out.println("\n=== " + UniversityConstants.UNIVERSITY_NAME + " Management
System ===");
    System.out.println("1. Add Student");
    System.out.println("2. Add Professor");
    System.out.println("3. Enroll Student in Course");
    System.out.println("4. Assign Course to Professor");
    System.out.println("5. Show Person Details");
    System.out.println("6. List All People");
    System.out.println("7. Exit");
    System.out.print("Enter choice: ");
    choice = sc.nextInt();
    sc.nextLine();

    switch (choice) {
        case 1: // Add Student
            System.out.print("Enter Student Name: ");
            String sName = sc.nextLine();
            System.out.print("Enter Student ID: ");
            int sId = sc.nextInt();
            sc.nextLine();
            people.add(new Student(sName, sId));
            System.out.println("Student added.");
            break;
        case 2: // Add Professor
            System.out.print("Enter Professor Name: ");
            String pName = sc.nextLine();
            System.out.print("Enter Professor ID: ");
            int pId = sc.nextInt();
            sc.nextLine();
            System.out.print("Enter Department: ");
            String dept = sc.nextLine();
            people.add(new Professor(pName, pId, dept));
            System.out.println("Professor added.");
            break;
        case 3: // Enroll Student
            System.out.print("Enter Student ID: ");
            int esId = sc.nextInt();
            sc.nextLine();
            boolean foundStudent = false;
            for (Person p : people) {
                if (p instanceof Student && p.id == esId) {
                    System.out.print("Enter Course to Enroll: ");
                    String course = sc.nextLine();
                    ((Enrollable)p).enroll(course);
                    foundStudent = true;
                    break;
                }
            }
            if (!foundStudent) {
                System.out.println("Student not found.");
            }
            break;
        case 4: // Assign Course to Professor
            System.out.print("Enter Professor ID: ");
            int epId = sc.nextInt();
            sc.nextLine();
            boolean foundProf = false;

```

```

        for (Person p : people) {
            if (p instanceof Professor && p.id == epId) {
                System.out.print("Enter Course to Assign: ");
                String course = sc.nextLine();
                ((Enrollable)p).enroll(course);
                foundProf = true;
                break;
            }
        }
        if (!foundProf) {
            System.out.println("Professor not found.");
        }
        break;
    case 5: // Show Person Details
        System.out.print("Enter Person ID: ");
        int pid = sc.nextInt();
        sc.nextLine();
        boolean found = false;
        for (Person p : people) {
            if (p.id == pid) {
                p.showDetails();
                found = true;
                break;
            }
        }
        if (!found) {
            System.out.println("Person not found.");
        }
        break;
    case 6: // List All People
        if (people.isEmpty()) {
            System.out.println("No people in the system.");
            break;
        }
        for (Person p : people) {
            p.showDetails();
            System.out.println("-----");
        }
        break;
    case 7:
        System.out.println("Exiting... Thank you!");
        break;
    default:
        System.out.println("Invalid choice!");
    }
} while (choice != 7);
sc.close();
}
}

```

```
PS D:\SEMESTER I\LAB WORK\Java Programming> cd "d:\SEMESTER I\LAB WORK\Java Programm
a } ; if ($?) { java UniversityManagement }
```

```
=== ABC University Management System ===
```

1. Add Student
2. Add Professor
3. Enroll Student in Course
4. Assign Course to Professor
5. Show Person Details
6. List All People
7. Exit

Enter choice: 1

Enter Student Name: Aakash

Enter Student ID: 1

Student added.

```
=== ABC University Management System ===
```

1. Add Student
2. Add Professor
3. Enroll Student in Course
4. Assign Course to Professor
5. Show Person Details
6. List All People
7. Exit

Enter choice: 1

Enter Student Name: Shivansh

Enter Student ID: 2

Student added.

```
=== ABC University Management System ===
```

1. Add Student
2. Add Professor
3. Enroll Student in Course
4. Assign Course to Professor
5. Show Person Details
6. List All People
7. Exit

Enter choice: 2

Enter Professor Name: Syed Hussain

Enter Professor ID: 100

Enter Department: Java

Professor added.

```
=== ABC University Management System ===
```

1. Add Student
2. Add Professor
3. Enroll Student in Course
4. Assign Course to Professor
5. Show Person Details
6. List All People
7. Exit

```
Enter choice: 3
```

```
Enter Student ID: 1
```

```
Enter Course to Enroll: Java
```

```
Aakash enrolled in Java
```

```
=== ABC University Management System ===
```

1. Add Student
2. Add Professor
3. Enroll Student in Course
4. Assign Course to Professor
5. Show Person Details
6. List All People
7. Exit

```
Enter choice: 4
```

```
Enter Professor ID: 100
```

```
Enter Course to Assign: Java
```

```
Syed Hussain assigned to teach Java
```

```
=== ABC University Management System ===
```

1. Add Student
2. Add Professor
3. Enroll Student in Course
4. Assign Course to Professor
5. Show Person Details
6. List All People
7. Exit

```
Enter choice: 5
```

```
Enter Person ID: 2
```

```
Student Name: Shivansh
```

```
Student ID: 2
```

```
Courses: []
```

```
=== ABC University Management System ===
```

1. Add Student
2. Add Professor
3. Enroll Student in Course
4. Assign Course to Professor
5. Show Person Details
6. List All People
7. Exit

```
Enter choice: 5
```

```
Enter Person ID: 1
```

```
Student Name: Aakash
```

```
Student ID: 1
```

```
Courses: [Java]
```

```
=== ABC University Management System ===
```

1. Add Student
2. Add Professor
3. Enroll Student in Course
4. Assign Course to Professor
5. Show Person Details
6. List All People
7. Exit

```
Enter choice: 5
```

```
Enter Person ID: 100
```

```
Professor Name: Syed Hussain
```

```
Professor ID: 100
```

```
Department: Java
```

```
Courses Teaching: [Java]
```

```
=== ABC University Management System ===
```

1. Add Student
2. Add Professor
3. Enroll Student in Course
4. Assign Course to Professor
5. Show Person Details
6. List All People
7. Exit

```
Enter choice: 6
```

```
Student Name: Aakash
```

```
Student ID: 1
```

```
Courses: [Java]
```

```
-----  
Student Name: Shivansh
```

```
Student ID: 2
```

```
Courses: []
```

```
-----  
Professor Name: Syed Hussain
```

```
Professor ID: 100
```

```
Department: Java
```

```
Courses Teaching: [Java]
```

```
-----  
=== ABC University Management System ===
```

1. Add Student
2. Add Professor
3. Enroll Student in Course
4. Assign Course to Professor
5. Show Person Details
6. List All People
7. Exit

```
Enter choice: 7
```

```
Exiting... Thank you!
```

```
PS D:\SEMESTER I\LAB WORK\Java Programming\lab8(2709)> █
```